```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit

spark = SparkSession.builder.appName("ChainingTransformations").getOrCreate()

data = [("Alice", 30, "NY", 75000),
        ("Bob", 25, "LD", 60000),
        ("Charlie", 35, "NY", 90000),
        ("David", 22, "SF", 55000),
        ("Eve", 40, "LD", 100000)]
columns = ["Name", "Age", "City_Code", "Salary"]
df = spark.createDataFrame(data, columns)
df.show()

# Chain multiple transformations:
# 1. Filter for age > 25
# 2. Add a new column 'Bonus'
# 3. Select specific columns and rename one
# 4. Show the result
df_processed = df.filter(col("Age") > 25) \
                 .withColumn("Bonus", col("Salary") * 0.10) \
                 .select(col("Name"),
                         col("Age").alias("YearsOld"),
                         col("Salary"),
                         col("Bonus"),
                         lit("Processed").alias("Status")) \
                 .sort(col("Salary").desc()) # Add a sort for good measure

print("Chained transformations result:")
df_processed.show()

spark.stop()
```

Creating calculated columns

Calculated columns are new columns derived from existing columns using expressions or functions.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, concat_ws, lit, when

```

```
spark = SparkSession.builder.appName("CalculatedColumns").getOrCreate()

data = [("Alice", "Smith", 30, 75000),
        ("Bob", "Johnson", 25, 60000),
        ("Charlie", "Brown", 35, 90000)]
columns = ["FirstName", "LastName", "Age", "Salary"]
df = spark.createDataFrame(data, columns)
df.show()

# 1. Calculate Full Name
df_full_name = df.withColumn("FullName", concat_ws(" ", col("FirstName"), col("LastName")))
print("After adding 'FullName':")
df_full_name.show()

# 2. Calculate Annual Bonus (10% of salary if age > 30, else 5%)
df_bonus = df.withColumn("AnnualBonus",
                         when(col("Age") > 30, col("Salary") * 0.10)
                         .otherwise(col("Salary") * 0.05))
print("After adding 'AnnualBonus':")
df_bonus.show()

# 3. Combine multiple calculations and selections
df_combined = df.withColumn("FullName", concat_ws(" ", col("FirstName"), col("LastName"))) \
                .withColumn("ExperienceCategory",
                            when(col("Age") < 28, "Junior")
                            .when(col("Age") < 35, "Mid-level")
                            .otherwise("Senior")) \
                .select("FullName", "Age", "Salary", "ExperienceCategory")
print("After combining multiple calculations:")
df_combined.show()

spark.stop()
```

Column Operations

Spark provides a rich set of functions in `pyspark.sql.functions` that operate on `Column` objects, enabling powerful data manipulations.

Using col, lit, expr, when, otherwise

These are fundamental functions for constructing expressions on DataFrames.

- `col(column_name)`: References a column by its name. It's the safest way to refer to columns, especially when column names might conflict with keywords or contain special characters.
- `lit(value)`: Creates a literal column with the given value. Useful for adding constant values to a DataFrame.
- `expr(sql_expression_string)`: Allows you to use SQL expressions directly within DataFrame operations. Very powerful for complex logic.
- `when(condition, value)`: Implements conditional logic (like `IF` or `CASE` in SQL). If `condition` is true, it returns `value`.
- `otherwise(value)`: Used in conjunction with `when()` to specify the default value if none of the `when()` conditions are met.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, expr, when, otherwise

spark = SparkSession.builder.appName("ColumnOperations").getOrCreate()

data = [("Alice", 75, "A"), ("Bob", 88, "B"), ("Charlie", 62, "C"), ("David", 95, "A")]
columns = ["Name", "Score", "Grade"]
df = spark.createDataFrame(data, columns)
df.show()

# Using col(): Select 'Name' and 'Score'
df.select(col("Name"), col("Score")).show()

# Using lit(): Add a 'Status' column with a constant value
df.withColumn("Status", lit("Processed")).show()

# Using expr(): Add a 'DoubleScore' column using SQL expression
df.withColumn("DoubleScore", expr("Score * 2")).show()

# Using when() and otherwise(): Create a 'PassStatus' column
df.withColumn("PassStatus", when(col("Score") >= 70, "Pass").otherwise("Fail")).show()

# Combining them:
df.select(
    col("Name"),
    col("Score"),
    lit("Exam").alias("AssessmentType"), # Add a literal column
    expr("Score / 100 * 100").alias("Percentage"), # Calculate percentage
    when(col("Grade") == "A", "Excellent") # Conditional logic based on Grade
    .when(col("Grade") == "B", "Good")
    .otherwise("Needs Improvement")
    .alias("Performance")
```

```
).show()

spark.stop()
```

Creating conditional logic

Conditional logic is essential for data cleaning, transformation, and feature engineering. `when()` and `otherwise()` are your go-to tools for this.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit

spark = SparkSession.builder.appName("ConditionalLogic").getOrCreate()

data = [("Alice", 18, 50000),
        ("Bob", 25, 60000),
        ("Charlie", 32, 80000),
        ("David", 40, 95000),
        ("Eve", 16, 40000)]
columns = ["Name", "Age", "Salary"]
df = spark.createDataFrame(data, columns)
df.show()

# Create a 'AgeGroup' column based on Age
df_with_age_group = df.withColumn("AgeGroup",
                                  when(col("Age") < 20, "Teenager")
                                  .when(col("Age") >= 20, "Adult")
                                  .otherwise("Senior")) # This 'otherwise' is technically redundant here
print("DataFrame with 'AgeGroup':")
df_with_age_group.show()

# Create a 'TaxBracket' column based on Salary
df_with_tax_bracket = df.withColumn("TaxBracket",
                                    when(col("Salary") < 50000, "Low")
                                    .when((col("Salary") >= 50000) & (col("Salary") < 80000), "Medium")
                                    .otherwise("High"))
print("DataFrame with 'TaxBracket':")
df_with_tax_bracket.show()

spark.stop()
```

Nesting conditions using multiple when

You can chain multiple `when()` clauses to create more complex conditional logic, similar to `CASE WHEN ... THEN ... WHEN ... THEN ... ELSE ... END` in SQL. The conditions are evaluated in order. The first `when()` condition that evaluates to true will have its corresponding value returned.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, lit

spark = SparkSession.builder.appName("NestedConditions").getOrCreate()

data = [("Alice", 85),
        ("Bob", 72),
        ("Charlie", 91),
        ("David", 60),
        ("Eve", 45)]
columns = ["Student", "Score"]
df = spark.createDataFrame(data, columns)
df.show()

# Assign grades based on score using nested when
df_with_grades = df.withColumn("Grade",
                               when(col("Score") >= 90, "A")
                               .when(col("Score") >= 80, "B")
                               .when(col("Score") >= 70, "C")
                               .when(col("Score") >= 60, "D")
                               .otherwise("F")) # Default if no other conditions met
print("DataFrame with assigned Grades:")
df_with_grades.show()

# More complex example: Eligibility for a scholarship
df_with_scholarship = df.withColumn("ScholarshipStatus",
                                    when((col("Score") >= 90) & (col("Student") == "Charlie"), lit("Full Scholarship"))
                                    .when(col("Score") >= 85, lit("Partial Scholarship"))
                                    .when(col("Score") >= 70, lit("Eligibility Review"))
                                    .otherwise(lit("Not Eligible")))
print("DataFrame with Scholarship Status:")
df_with_scholarship.show()

```

- spark.stop()

Data Types & Schema

Understanding and managing data types and schemas is fundamental for robust data engineering with Spark.

StructType, StructField, ArrayType, MapType

Spark's `pyspark.sql.types` module provides classes to define schemas programmatically.

- **`StructType`**: Represents the schema of a DataFrame, which is a list of `StructField` objects. It defines the structure of a row.
- **`StructField`**: Represents a column within a `StructType`. It takes three arguments:
  - `name` (string): The name of the column.
  - `dataType` (DataType): The data type of the column (e.g., `StringType()`, `IntegerType()`, `BooleanType()`).
  - `nullable` (boolean): Whether the column can contain null values. `True` if nullable, `False` otherwise.
- **`ArrayType(elementType, containsNull)`**: Represents an array (list) of elements of a specific type.
  - `elementType`: The data type of the elements in the array.
  - `containsNull`: Whether the array can contain null elements.
- **`MapType(keyType, valueType, valueContainsNull)`**: Represents a map (dictionary) with key-value pairs.
  - `keyType`: The data type of the keys.
  - `valueType`: The data type of the values.
  - `valueContainsNull`: Whether the map values can be null.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, ArrayType, MapType

spark = SparkSession.builder.appName("DataTypesSchema").getOrCreate()

# Define a complex schema
schema = StructType([
    StructField("employee_id", IntegerType(), False), # Not nullable
    StructField("name", StructType([              # Nested StructType
        StructField("first", StringType(), True),
        StructField("last", StringType(), True)
```

```
    ]), True),
    StructField("skills", ArrayType(StringType(), True), True), # Array of strings
    StructField("contact_info", MapType(StringType(), StringType(), True), True) # Map with string keys and values
])

# Create data that conforms to the schema
data = [(1, ("Alice", "Smith"), ["Python", "Spark"], {"email": "alice@example.com", "phone": "123-456-7890"}),
        (2, ("Bob", "Johnson"), ["Java", "SQL"], {"email": "bob@example.com"}),
        (3, ("Charlie", None), [], {}) # Example with null in nested struct, empty array, empty map
       ]

df = spark.createDataFrame(data, schema)

print("DataFrame with complex schema:")
df.show(truncate=False)
df.printSchema()

spark.stop()
```

---

inferSchema vs defining schema manually

When reading data from external files, Spark can either infer the schema or you can provide it manually.

- **`inferSchema=True` (Automatic Schema Inference):**
  - Spark samples a portion of the data to guess the column names and data types.
  - **Pros:** Convenient, less manual work, good for quick exploration.
  - **Cons:** Can be slow (requires an extra pass over the data). May infer incorrect types (e.g., all strings if data is dirty, or integer instead of string for IDs). Not suitable for production pipelines where schema stability is critical.
  - **Usage:** `spark.read.csv("path/to/file.csv", header=True, inferSchema=True)`
- **Defining Schema Manually:**
  - You explicitly define the `StructType` and `StructField` for each column.
  - **Pros:** Fast (no extra pass for inference). Robust (prevents unexpected type changes due to data variations). Enables early detection of schema mismatches. Recommended for production ETL.
  - **Cons:** More verbose, requires knowing the schema beforehand.
  - **Usage:** `spark.read.csv("path/to/file.csv", header=True, schema=my_schema)`

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName("InferVsManualSchema").getOrCreate()

# Create a dummy CSV file
csv_content = """id,name,age,city
1,Alice,30,New York
2,Bob,25,London
3,Charlie,35,Paris
4,David,NULL,Berlin
"""
with open("people_data.csv", "w") as f:
    f.write(csv_content)

# 1. Infer Schema
df_inferred = spark.read.csv("people_data.csv", header=True, inferSchema=True)
print("DataFrame with Inferred Schema:")
df_inferred.printSchema()
df_inferred.show()

# Notice 'age' might be inferred as IntegerType, but if 'NULL' is present it might become StringType or FloatType for missing values.

# 2. Define Schema Manually
manual_schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True), # Define as IntegerType, expecting nulls
    StructField("city", StringType(), True)
])

df_manual = spark.read.csv("people_data.csv", header=True, schema=manual_schema)
print("\nDataFrame with Manual Schema:")
df_manual.printSchema()
df_manual.show()

spark.stop()
```

---

cast() for type conversions

The `cast()` function is used to convert a column from one data type to another. This is crucial for data cleaning and ensuring data integrity.

Example (Python):

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.sql.types import IntegerType, StringType, DoubleType, DateType

spark = SparkSession.builder.appName("TypeConversions").getOrCreate()

data = [("1", "10.5", "2023-01-15"),
        ("2", "20.7", "2023-02-20"),
        ("3", "30.9", "2023-03-25")]
columns = ["id_str", "price_str", "date_str"]
df = spark.createDataFrame(data, columns)
print("Original DataFrame Schema:")
df.printSchema()
df.show()

# Cast 'id_str' to IntegerType
df_casted = df.withColumn("id_int", col("id_str").cast(IntegerType()))

# Cast 'price_str' to DoubleType
df_casted = df_casted.withColumn("price_double", col("price_str").cast(DoubleType()))

# Cast 'date_str' to DateType
df_casted = df_casted.withColumn("date_date", col("date_str").cast(DateType()))

print("\nDataFrame After Casts:")
df_casted.printSchema()
df_casted.show()

# Handling invalid casts:
# If a value cannot be cast, it will result in a null.
data_invalid = [("1", "abc"), ("2", "123")]
df_invalid = spark.createDataFrame(data_invalid, ["id", "num_str"])
print("\nDataFrame with potential invalid casts:")
df_invalid.show()
df_invalid.withColumn("num_int", col("num_str").cast(IntegerType())).show()

spark.stop()
```

Schema evolution awareness

Schema evolution refers to the ability to handle changes in the schema of data over time. This is a common challenge in big data systems, especially when dealing with semi-structured data sources or streaming data.

Spark, particularly when working with formats like Parquet (which has built-in schema evolution support), is generally good at handling schema evolution.

Common Scenarios and How Spark Handles Them:

1. **Adding new columns:** If new columns are added to the source data, Spark will typically include them in the DataFrame's schema. Older records (without the new columns) will have `null` values for those columns.
2. **Dropping columns:** If columns are removed from the source, Spark will ignore them if you read with an existing schema, or the schema will update if inferred.
3. **Changing data types:** This is the trickiest. If a data type changes in a way that is compatible (e.g., `Integer` to `Long`), Spark might handle it. However, incompatible changes (e.g., `String` to `Integer` where strings aren't valid numbers) will likely lead to `null` values or errors.
4. **Reordering columns:** Spark (especially Parquet) generally handles column reordering gracefully as it identifies columns by name, not by position.

Best Practices for Schema Evolution:

- **Use Parquet or Avro:** These formats are designed with schema evolution in mind. They store schema information with the data and can handle additive changes efficiently.
- **Define Schema Manually (when possible):** While `inferSchema` is convenient, explicitly defining your schema gives you more control and helps catch unexpected schema changes early.
- **Handle Nulls:** Be prepared for `null` values when new columns are added.
- **Version Control Schemas:** In complex data pipelines, consider versioning your schemas to track changes.
- **Monitor Data Quality:** Implement data quality checks to detect unexpected schema changes or data type issues.

Example (Conceptual):

Imagine you have a Parquet file with (id INT, name STRING).

Later, new data arrives with (id INT, name STRING, age INT).

When you read the combined data with Spark, the resulting DataFrame will have (id INT, name STRING, age INT). The older records will have null for age.

This "awareness" means you, as the data engineer, should anticipate these changes and design your pipeline to be resilient to them.

Reading & Writing Files